



Forest Crawler - Documentation

MABROUK Sofia, SAYAVONG Melvin

ENSEA, 2025 - 2026, Semestre 6

Table des matières

1	Présentation	2
2	Class Main (peut-être continuer)	2
3	Class Game Engine	2
3.1	Variables	2
3.2	keyTyped	2
3.3	keyPressed	3
3.4	keyReleased	3
4	RenderEngine	3
4.1	Variables	3
4.1.1	Screen settings	3
4.1.2	World settings	4
4.1.3	FPS	4
4.2	RenderEngine	4
4.3	startGameThread	4
5	Class Player	4
5.1	Variables (description + à compléter)	4
5.2	Player	5
5.3	setDefaultValues	5
5.4	getPlayerImage	5
5.5	update (mettre image)	5
5.6	draw (mettre une image)	6
6	Class Sprite	6
7	Class CollisionCheck	6
7.1	CollisionCheck	6
7.2	checkTile (à faire)	6
8	Tile	6
9	TileManager (à faire)	7

1 Présentation

2 Class Main (peut-être continuer)

La classe Main est la classe permettant d'exécuter tout le jeu, il s'occupe principalement de la gestion de la fenêtre.

```
/*Game window configuration*/
JFrame window = new JFrame();
window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
window.setResizable(false);
window.setTitle("Forest Crawler");
```

Cette partie du code correspond aux paramètres de la fenêtre :

- `window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`: permet de déterminer le comportement du bouton de fermeture de la fenêtre.
- `window.setResizable(false)`: permet de savoir si la taille de la fenêtre est modifiable ou non.
- `window.setTitle("Forest Crawler")`: correspond au nom de la fenêtre.

3 Class Game Engine

Cette classe s'occupe de la gestion du comportement des touches à l'aide d'un listener. Elle sert uniquement à configurer les touches pour le déplacement du joueur.

3.1 Variables

Cette classe permet de gérer le comportement des touches du clavier à l'aide d'un listener. Dans notre cas, on utilise les touches du clavier uniquement pour se déplacer donc on crée 4 variables booléennes :

- `upPressed`
- `downPressed`
- `leftPressed`
- `rightPressed`

3.2 keyTyped

```
@Override  @sofia
public void keyTyped(KeyEvent keyEvent) {

}
```

Cette méthode permet de savoir si une touche a été appuyé puis relâché. Elle ne sert pas pour ce projet, mais est automatiquement créée par IntelliJ dans la classe KeyListener (qui est implémentée par GameEngine) avec `keyPressed` et `keyReleased`.

3.3 keyPressed

```
@Override
public void keyPressed(KeyEvent keyEvent) {
    int code = keyEvent.getKeyCode();

    if(code == KeyEvent.VK_Z){
        upPressed = true;
    }
    if(code == KeyEvent.VK_Q){
        leftPressed = true;
    }
    if(code == KeyEvent.VK_S){
        downPressed = true;
    }
    if(code == KeyEvent.VK_D){
        rightPressed = true;
    }
}
```

Cette méthode permet de savoir si les touches liées au déplacement ont été appuyées.

3.4 keyReleased

```
@Override
public void keyReleased(KeyEvent keyEvent) {
    int code = keyEvent.getKeyCode();

    if(code == KeyEvent.VK_Z){
        upPressed = false;
    }
    if(code == KeyEvent.VK_Q){
        leftPressed = false;
    }
    if(code == KeyEvent.VK_S){
        downPressed = false;
    }
    if(code == KeyEvent.VK_D){
        rightPressed = false;
    }
}
```

Cette méthode permet de savoir si les touches liées au déplacement ont été relâchées.

4 RenderEngine

Cette classe correspond à la configuration du rendu du jeu au sein de la fenêtre comme la taille des éléments du jeu. Ce rendu dépend de deux groupes de paramètres de position : **screenXXX**, une position relative à la fenêtre de jeu, et **worldXXX**, une position par relative à la map du jeu, qui est 'out of bounds' de l'écran.

4.1 Variables

4.1.1 Screen settings

Voici la liste des différentes variables pour cette section :

- **originalTileSize** : Taille réelle des cases
- **scale** : Échelle d'agrandissement
- **tileSize** : Taille des cases lors de l'affichage

- `maxScreenCol` : Nombre de colonnes affichées sur la fenêtre de jeu
- `maxScreenRow` : Nombre de lignes affichées sur la fenêtre de jeu
- `screenWidth` : Longueur de la fenêtre de jeu
- `screenHeight` : Hauteur de la fenêtre de jeu

4.1.2 World settings

Voici la liste des différentes variables pour cette section :

- `maxWorldCol` : Nombre de colonnes de la map de jeu
- `maxWorldRow` : Nombre de lignes de la map de jeu
- `worldWidth` : Longueur maximum de la carte affichée
- `worldHeight` : Hauteur maximum de la carte affichée

4.1.3 FPS

La variable FPS correspond au nombre d'images par secondes.

4.2 RenderEngine

```
public RenderEngine(){ 1 usage  @sofia
    this.setPreferredSize(new Dimension(screenWidth , screenHeight));
    this.setBackground(Color.decode( nm: "#55af32"));
    this.setDoubleBuffered(true); /*All the drawings from this component will be done in an offscreen painting buffer
    which should improve the game's graphic performances*/
    this.addKeyListener(keyH); /*Engine can recognize key input*/
    this.setFocusable(true );
}
```

Cette méthode permet de définir les différents paramètres de la fenêtre.

4.3 startGameThread

```
public void startGameThread(){ 1 usage  @sofia
    gameThread = new Thread( task: this);
    gameThread.start();
}
```

Cette méthode permet d'initialiser le thread pour le jeu.

5 Class Player

Cette classe permet de gérer tous les attributs et comportements liés au joueur comme sa position, son déplacement ou son sprite.

5.1 Variables (description + à compléter)

- `RenderEngine re` : instance de `RenderEngine`
- `GameEngine keyH` : instance de `GameEngine`
- `screenX` : position X du joueur sur la carte.
- `screenY` : position Y du joueur sur la carte.

5.2 Player

```
public Player(RenderEngine re, GameEngine keyH) { 1 usage  🧑 sofia *
    this.re = re;
    this.keyH = keyH;

    screenX = re.screenWidth/2 - re.tileSize/2;
    screenY = re.screenHeight/2 - re.tileSize/2;

    solidArea = new Rectangle(x: 16, y: 30, width: 32, height: 32);

    setDefaultValues();
    getPlayerImage();
}
```

Cette méthode correspond au constructeur de l'objet Player

- `solidArea` : Rectangle placé sur le joueur, qui sert de zone de collision (cf. CollisionCheck)
- `screenX`, `screenY` : Positionnement du joueur au centre de la fenêtre de jeu

5.3 setDefaultValues

```
public void setDefaultValues(){ 1 usage  🧑 sofia
    worldX= re.tileSize * 27;
    worldY = re.tileSize * 25 ;
    speed=4;
    direction="down";
}
```

Cette méthode permet de définir les valeurs par défaut lorsqu'on lance le jeu.

5.4 getPlayerImage

```
public void getPlayerImage(){ 1 usage  🧑 sofia
    try{
        up1 = ImageIO.read(getClass().getResourceAsStream( name: "/characters/gnome/walk/gnome_z_walk1.png"));
        up2 = ImageIO.read(getClass().getResourceAsStream( name: "/characters/gnome/walk/gnome_z_walk2.png"));
        down1 = ImageIO.read(getClass().getResourceAsStream( name: "/characters/gnome/walk/gnome_s_walk1.png"));
        down2= ImageIO.read(getClass().getResourceAsStream( name: "/characters/gnome/walk/gnome_s_walk2.png"));
        right1 = ImageIO.read(getClass().getResourceAsStream( name: "/characters/gnome/walk/gnome_d_walk1.png"));
        right2 = ImageIO.read(getClass().getResourceAsStream( name: "/characters/gnome/walk/gnome_d_walk2.png"));
        left1 = ImageIO.read(getClass().getResourceAsStream( name: "/characters/gnome/walk/gnome_q_walk1.png"));
        left2 = ImageIO.read(getClass().getResourceAsStream( name: "/characters/gnome/walk/gnome_q_walk2.png"));
    }catch(IOException e){
        e.printStackTrace();
    }
}
```

Cette méthode permet de chercher les sprites du joueur en fonction de sa direction. Il y a 2 sprites par direction pour donner l'illusion du mouvement lors du déplacement.

5.5 update (mettre image)

Cette méthode permet de vérifier à tout instant les événements de déplacement et de collision ainsi que de mettre à jour le sprite.

5.6 draw (mettre une image)

Cette méthode permet de mettre à jour la bonne image en fonction de la direction, en animant le mouvement.

6 Class Sprite

Cette classe permet de configurer le comportement des sprites.

```
public class Sprite { 2 usages 1 inheritor 2 sofia

    public int worldX, worldY ; 8 usages
    public int speed; 9 usages
    public BufferedImage up1, up2, down1, down2, right1, right2, left1, left2, idle1, idle2;
    public String direction; 8 usages

    public int spriteCounter = 0; 3 usages
    public int spriteNumber = 1; 12 usages

    public Rectangle solidArea; 11 usages
    public boolean collisionOn = false; 6 usages
}
```

Voici la description des différentes variables :

- **worldX, worldY** : Position X et Y du sprite sur la carte
- **speed** : Vitesse du sprite sur la carte
- **up, down, right, left, idle** : Variables permettant d'utiliser les bonnes images en fonction de la direction
- **direction** : Direction du sprite parmi [up, down, left, right]
- **spriteCounter** : Compteur permettant d'actualiser les images pour l'animation
- **spriteNumber** : Permet de basculer entre les différentes images lors d'une animation
- **solidArea** : Hitbox du sprite
- **collisionOn** : Activation de la collision

7 Class CollisionCheck

Cette classe permet de faire la gestion des collisions avec les différents sprite.

7.1 CollisionCheck

Constructeur pour définir le RenderEngine.

7.2 checkTile (à faire)

8 Tile

Cette classe permet de gérer l'affichage des cases ainsi que le comportement de celle-ci vis-à-vis du joueur. La class Tile contient 4 variables :

- **image** : prends l'image correspondante
- **collision** : Permet de définir si l'image devra être une collision ou non
- **isOverhead** : Permet de définir si l'image devra être au dessus du joueur ou non, (cf. méthode draw de TileManager)
- **width, height** : Définit la taille de la case

9 TileManager (à faire)